

Министерство науки и высшего образования Российской Федерации

ФГБОУ ВО АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Институт цифровых технологий, электроники и физики

Кафедра вычислительной техники и электроники (ВТиЭ)

Отчёт по практике на тему:

РАЗРАБОТКА TELEGRAM БОТА НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ
PYTHON

Выполнил студент 5.205-1 группы:

_____ Д. А. Кулиев

«___» _____ 2024 г.

Проверил: ст.пр

_____ И. А. Шмаков

«___» _____ 2024 г.

Барнаул 2024 г.

РЕФЕРАТ

Полный объём работы составляет 27 страниц, включая 0 рисунков и 0 таблиц.

В отчёте содержатся сведения о курсовой работе.

Курсовая работа заключается в создании Telegram бота.

В отчёте приведено описание используемой библиотеки и инструментария для написания бота на языке программирования Python.

Также в отчёте приведены общие сведения об Telegram боте.

Наряду с этим представлены проверка работоспособности бота, диаграмма UML, полный код программы.

Ключевые слова: Телеграм, Телеграм бот, бот.

Отчёт оформлена с помощью системы компьютерной вёрстки $\text{T}_\text{E}\text{X}$ и его расширения $\text{X}_\text{E}\text{L}_\text{A}\text{T}_\text{E}\text{X}$ из дистрибутива *TeX Live*.

ABSTRACT

The total amount of work is 27 page's, include 0 image's and 0 table's.

The course work is to create a Telegram bot.

The report describes the library and tools used to write a bot in the Python programming language.

The report also provides general information about the Telegram bot.

Along with this, a bot health check, a UML diagram, and the full program code are presented.

Keyword: Telegram, Telegram bot, bot.

Report is framed using the computer layout system $\text{T}_\text{E}\text{X}$ and its extension $\text{X}_\text{E}\text{L}_\text{A}\text{T}_\text{E}\text{X}$ from the distribution *TeX Live*.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ОБЗОР ИНСТРУМЕНТАРИЯ ДЛЯ СОЗДАНИЯ TELGRAM BOT .	6
1.1. Техническое задание.	6
1.2. Библиотека Aiogram.	6
1.3. Инструментарий.	7
1.4. Telegram Bot.	8
2. ЭТАПЫ РАЗРАБОТКИ Бота	10
2.1. Создание Бота.	10
2.2. Реализация команд ЧАСТЬ 1.	10
2.3. Реализация авторизации.	10
2.4. Реализация команд ЧАСТЬ 2.	11
2.5. Блок-схема.	13
2.6. Проверка работоспособности программы.	15
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	17
Приложение	19

ВВЕДЕНИЕ

Актуальность: Разработка телеграмм бота представляет собой актуальную и востребованную тему в современном цифровом мире по нескольким причинам.

1. Возрастающая популярность мессенджеров Telegram является одним из самых популярных мессенджеров в мире, с активно растущей базой пользователей. Миллионы людей ежедневно используют данную платформу для общения, обмена информацией и решения рабочих задач. В связи с этим появление и развитие телеграмм ботов становится логичным шагом для улучшения пользовательского опыта и автоматизации рутинных процессов.

2. Автоматизация и оптимизация Боты могут выполнять множество задач, начиная от предоставления информации и заканчивая осуществлением сложных операций, таких как бронирование билетов, обработка платежей или управление проектами. Это позволяет бизнесам экономить ресурсы, улучшать качество обслуживания клиентов и ускорять рабочие процессы.

3. Улучшение пользовательского опыта Боты могут работать 24/7, предоставляя мгновенную помощь и поддержку пользователям. Это значительно повышает удовлетворенность пользователей, так как они могут получить ответы на свои вопросы без задержек и неограниченное время суток.

4. Возможность интеграции Телеграмм боты могут интегрироваться с различными сервисами и платформами, делая их незаменимыми инструментами для бизнеса. Они могут взаимодействовать с CRM-системами, базами данных, интернет-магазинами и другими ресурсами, обеспечивая комплексное решение многих задач.

5. Инновации и тренды Создание телеграмм ботов стало не только необходимостью, но и трендом. Внедрение ботов в бизнес-процессы и повседневную жизнь является примером инновационного подхода и свидетельствует о компании как об идущей в ногу со временем, что может позитивно сказываться на её репутации и конкурентоспособности.

6. Простота разработки и доступность Создание телеграмм бота не требует больших затрат и сложных технических навыков. Благодаря обширной документации и большим сообществам разработчиков, процесс создания бота стал более доступным, что привлекает как начинающих программистов, так и мелкий бизнес.

Таким образом, разработка телеграмм бота является актуальной темой, так как соответствует современным тенденциям цифровой трансформации, повышает эффективность и доступность услуг для пользователей, а также способствует инновациям и улучшению бизнес-процессов.

Цель: Создать Telegram бота с использованием языка программирования Python и библиотеки Aiogram.

Задачи:

1. Изучить библиотеку Aiogram.
2. Изучить дополнительные библиотеки для реализации задач поставленных боту.
3. Реализовать функционал бота.
4. Изучить принцип внедрения базы данных в Telegram бота.

1. ОБЗОР ИНСТРУМЕНТАРИЯ ДЛЯ СОЗДАНИЯ TELEGRAM BOT

1.1. Техническое задание.

1. Создание Telegram бота на языке Python.
2. Реализовать авторизацию.
3. Разработчик должен являться администратором и без изменения кода повышать или понижать привелегии пользователям.
4. Реализовать следующие команды(в скобках помечена категория пользователей, которая может использовать данные команды):
 - 4.1. Вывод информации о боте(все пользователи).
 - 4.2. Вывод расписания группы на текущий день(все пользователи).
 - 4.3. Вывод ссылки на сайт Wikipedia, по запросу пользователя(авторизованные пользователи).
 - 4.4. Вывод трёх поисковых запросов с сайта stackoverflow по теме JavaScript(авторизованные пользователи).
 - 4.5. Авторизовать и деавторизовать пользователей(администратор).
 - 4.6. Просмотр авторизованных пользователей(администратор).

1.2. Библиотека Aiogram.

Для создания Telegram бота на языке программирования Python была использована библиотека Aiogram [1].

Aiogram — это мощная и удобная библиотека для разработки ботов для Telegram на Python. Она основана на асинхронном подходе с использованием `async` и `await`, что позволяет создавать высокопроизводительные и масштабируемые приложения. Ниже приведено подробное описание основных аспектов Aiogram.

Основные особенности Aiogram:

1. Асинхронность: Aiogram использует библиотеку `asyncio` для обработки асинхронных операций. Это позволяет создавать ботов, которые могут

обработывать множество запросов одновременно, что особенно важно для ботов с большой нагрузкой.

2. Поддержка всех возможностей Telegram Bot API: Библиотека полностью поддерживает все функции Telegram Bot API, включая обработку сообщений, inline-режим, кнопки, обработку вложений и многое другое.
3. Простота использования: Aiogram имеет четкий и интуитивно понятный интерфейс. Это делает библиотеку доступной для новичков, а также предоставляет достаточную гибкость для опытных разработчиков.
4. Поддержка Middleware: Aiogram позволяет использовать middleware — промежуточные обработчики, которые могут выполнять действия до и после обработки входящих сообщений (например, аутентификация, логирование и другое).
5. Routing (Маршрутизация): Библиотека включает систему маршрутизации, что упрощает организацию кода и добавление новых функций в бота. Вы можете легко определять обработчики для разных типов сообщений и событий.
6. Удобные команды и обработчики: Aiogram позволяет легко обрабатывать команды пользователей и создавать свой собственный набор команд с помощью декораторов.
7. Работа с базами данных: Библиотека поддерживает интеграцию с различными базами данных. Обычно используется SQLAlchemy для работы с реляционными базами данных, а также другие ORM для взаимодействия с NoSQL базами.

1.3. Инструментарий.

Для написания отчёта с помощью системы компьютерной верстки в $\text{\LaTeX}$ был использован Online LaTeX Editor Overleaf.

Для написания кода программы был использован редактор Sublime Text3 [2].

Для работы с базой данных, используемой в ходе разработки программы, был использован DB Browser (SQLite).

Для хранения проекта была выбрана система контроля версия GitHub [3].

Проверка работоспособности и сборка программы выполнялась на системе:

- **ОС:** *Windows 10*
- **ЦП:** *Intel Pentium CPU G4600*
- **ОЗУ:** *16gb*
- **Видеокарта:** *NVIDIA GeForce 1650*

1.4. Telegram Bot.

Telegram Bot — это программа, которая работает внутри мессенджера Telegram и взаимодействует с пользователями через текстовые сообщения, кнопки, опросы и другие элементы интерфейса. Боты могут выполнять различные функции и задачи, автоматизируя процессы и улучшая взаимодействие с пользователями.

Основная цель Telegram Bot заключается в автоматизации взаимодействия с пользователями. Боты могут выполнять множество задач, начиная от простых напоминаний и заканчивая сложными системами поддержки пользователей. Они часто используются для:

1. Автоматизации служб: Например, предоставление информации о погоде, новостях, курсах валют и т.д.
2. Обслуживания клиентов: Ответы на часто задаваемые вопросы, предоставление информации о продуктах и услугах.
3. Игры и развлечения: Участие пользователей в играх, викторинах и других развлекательных активностях.
4. Управления задачами: Добавление задач в списки, напоминания и управление проектами.

5. Взаимодействия с другими сервисами: Интеграция с API различных сервисов для выполнения определённых действий (например, бронирование билетов, заказ еды и т.д.).

Функции Telegram Bot.

Боты могут выполнять множество функций:

1. Получение и отправка сообщений: Боты могут как отправлять текстовые сообщения, так и реагировать на сообщения пользователей.
2. Кнопки и меню: Боты могут предоставлять кнопки для выбора опций, создания инлайн-меню и использования клавиатур.
3. Обработка команд: Боты могут реагировать на команды пользователей, такие как /start, /help, и другие пользовательские команды.
4. Взаимодействие с API: Боты могут делать HTTP-запросы к другим API и возвращать данные пользователю.
5. Файлы и медиа: Боты могут отправлять и принимать изображения, видео, документы и другие файлы.
6. Оповещения и уведомления: Боты могут уведомлять пользователей о событиях, изменениях или напоминаниях.

Способы взаимодействия бота с Telegram

1. Webhook: Бот может использовать вебхуки для получения обновлений от Telegram. Это означает, что Telegram будет отправлять POST-запросы на указанный URL, когда происходят взаимодействия с ботом (например, пользователи отправляют сообщения). Этот способ требует настроенной веб-серверной части.
2. Polling: В этом случае бот регулярно запрашивает Telegram API на наличие обновлений. Это может быть проще в установке, но менее эффективным по сравнению с вебхуками, так как создает больше трафика и задержек.

2. ЭТАПЫ РАЗРАБОТКИ БОТА

2.1. Создание Бота.

Для начала я создал бота, используя BotFather.

BotFather [4] — это официальный бот в Telegram, который используется для создания и управления другими ботами. Он предоставляет пользователям простой интерфейс для регистрации новых ботов и настройки их параметров.

По итогу моему боту присвоился токен, который служит ключом для аутентификации моего бота в API Telegram. Он необходим для выполнения различных операций и управления поведением бота. Но мой бот был пуст, он не умел выполнять команды, поэтому далее я начал реализовывать команды, которые были даны в ТЗ.

2.2. Реализация команд ЧАСТЬ 1.

Первые команды, которые я реализовал была команда start и about.

1. Команда start отправляла основную, но краткую информацию о боте.
2. Команда about отправляла полную информацию о боте и его функционале.

2.3. Реализация авторизации.

Бот распознает три группы пользователей: администратор, авторизованный пользователь и обычный пользователь.

1. Администратор. Для того чтобы мой бот мог различать привилегию пользователя я написал фильтр. Суть фильтра была проста, бот ищет id пользователя, который ему пишет, в списке id админов и определяет привилегию пользователя. Если id нашлось в списке, то бот выполняет запрос пользователя.
2. Авторизованные пользователи. Сначала я хотел сделать такой же фильтр как и для администратора, но при реализации моей идеи, я понял, что обычный список не подойдет т.к. с каждым запуском бота он

будет обнуляться. Поэтому я принял решение создать базу данных, в которой будут храниться все id авторизованных пользователей. В итоге я использовал библиотеку sqlalchemy [5], создал и настроил свою БД. Теперь я мог обращаться к ней в любой момент для того чтобы записать новый id или удалить не нужный, не меняя код программы.

3. Неавторизованные пользователи. Для неавторизованного пользователя, было всё просто, ему был доступен ограниченный список команд и не было доступа к другим командам.
4. Повышение привилегии. Если пользователь захотел повысить свою привилегию, то он должен определить своё id с помощью команды my-id, которая доступна для всех групп пользователей и отправить администратору бота с запросом на повышение привилегии.

После того как я научил бота различать группы пользователей, я продолжил реализацию команд.

2.4. Реализация команд ЧАСТЬ 2.

Реализация команд для трёх групп пользователей.

1. Реализация команд для администратора была самая простая т.к я уже подключил БД. Всё что мне нужно было это создать выполнение трёх запросов: записать id, удалить и вывести все. В этом у меня не возникло трудностей. Всё что мне понадобилось дополнительно изучить это FSM модели. FSM (Finite State Machine или конечный автомат) в контексте телеграм-ботов — это модель, которая используется для управления состояниями и переходами между ними в процессе взаимодействия пользователя с ботом. Это особенно полезно при разработке диалоговых систем, где требуется поддерживать контекст общения. Например бот спрашивает у админа, какое id вы хотите удалить из БД и входит в режим ожидания, после, бот сохраняет ответ пользователя и использует полученную информацию для дальнейшего выполнения команды.
2. Реализация команд для авторизованных пользователей познакомила меня с парсингом. Парсинг сайтов — это процесс автоматизированно-

го извлечения данных с веб-страниц. Этот подход позволяет собирать информацию из различных источников в интернете для дальнейшего анализа, обработки или хранения. Парсинг часто используется в самых разных областях, таких как мониторинг цен, анализ конкурентов, сбор новостей и другие. После изучения основ парсинга с помощью библиотек BeautifulSoup [6] и requests [7].

Основной аспект парсинга сайтов, которым я пользовался это - HTTP-запросы: Чтобы получить данные с веб-страницы, парсер отправляет HTTP-запросы к серверу. Обычно используется метод GET, который позволяет запрашивать документы (веб-страницы) с определённого URL.

В итоге, я реализовал вывод ссылки на статью в Wikipedia [8] по запросу пользователя и вывод трёх статей с ссылками с сайта stackoverflow, также по запросу пользователя, только в этом случае, я использовал библиотеку json [9], которую описывали в самом api stackoverflow.

3. Для обычных пользователей мне нужно было тоже запарсить сайт Агу с расписанием нашей группы на текущий день. Сначала я разобрался с определением текущей даты, используя библиотеку datetime [10], отформатировал в нужный формат и получил заветные 8 цифр (например 20240912) и подставив это в нужную ссылку я переходил на страничку с сегодняшним расписанием. После чего мне оставалось сделать запрос к сайту и достать оттуда нужную информацию и тут у меня возникла проблема. При запросе к серверу, он возвращал мне html-код без блока с расписанием, хотя если исследовать код элемента в самом браузере, я без проблем находил блок с кодом расписания. После я обратился к преподавателю за помощью и нашли решение. Оказывается, что html-код появлялся после работы java-скрипта, который стоял перед ним. Моё решение было в использование библиотеки Selenium [11]. Благодаря возможности взаимодействовать с элементами на страницах, которые за-

гружаются динамически (например, через JavaScript), Selenium позволяет проверять функциональность современных веб-приложений.

Так я реализовал все команды из ТЗ. Далее сделал для них клавиатуру в моем боте и меню. После чего добавил команды help для всех видов пользователей.

2.5. Блок-схема.

Поскольку выше я описал принцип работы программы, то решил сделать блок-схему более абстрагированной ввиду громоздкости кода программы. Блок-схема, описывающая алгоритмы и процессы в программе, приведена на рисунках 2.1 2.2 2.3.

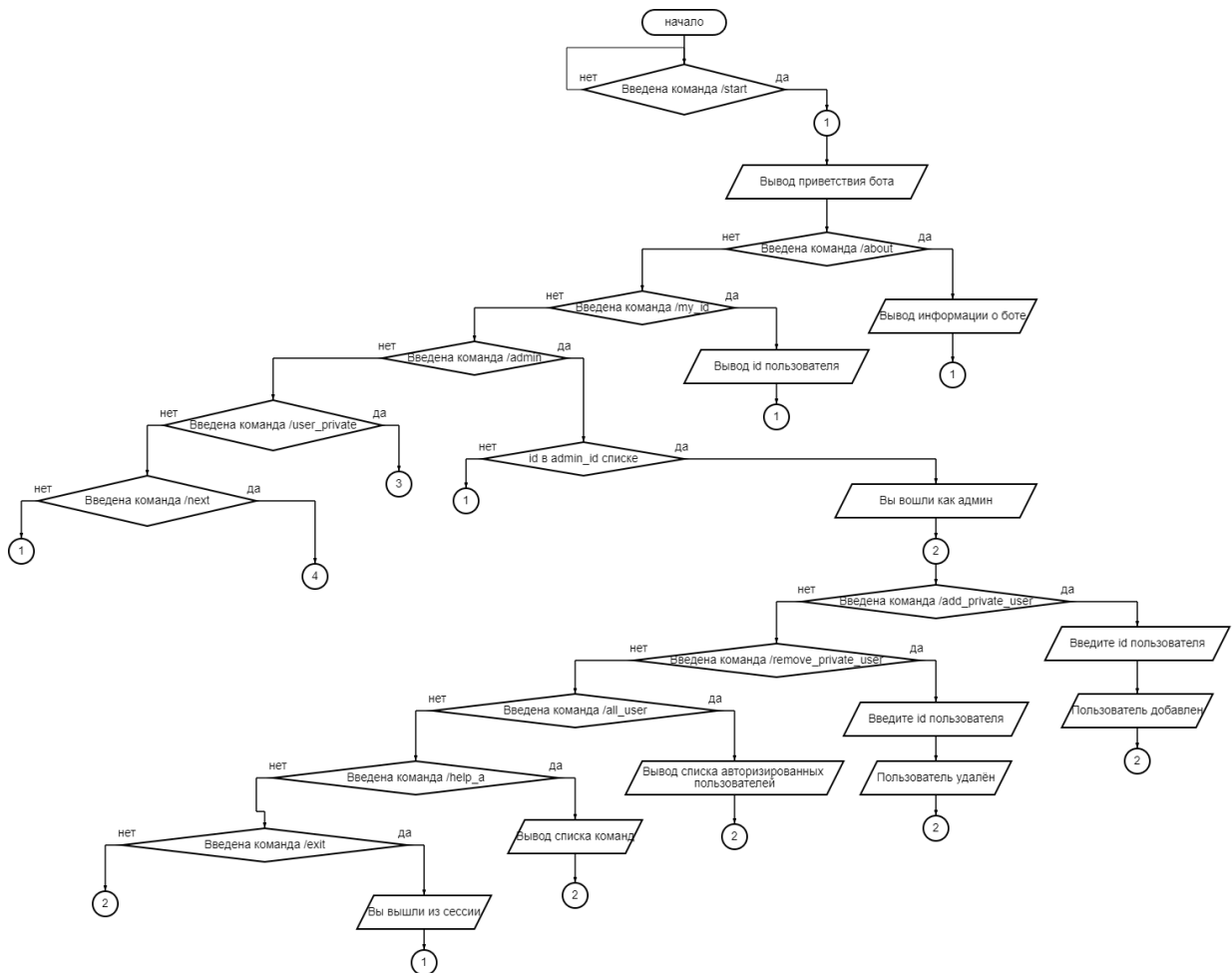


Рис. 2.1 Блок-схема алгоритма программы.

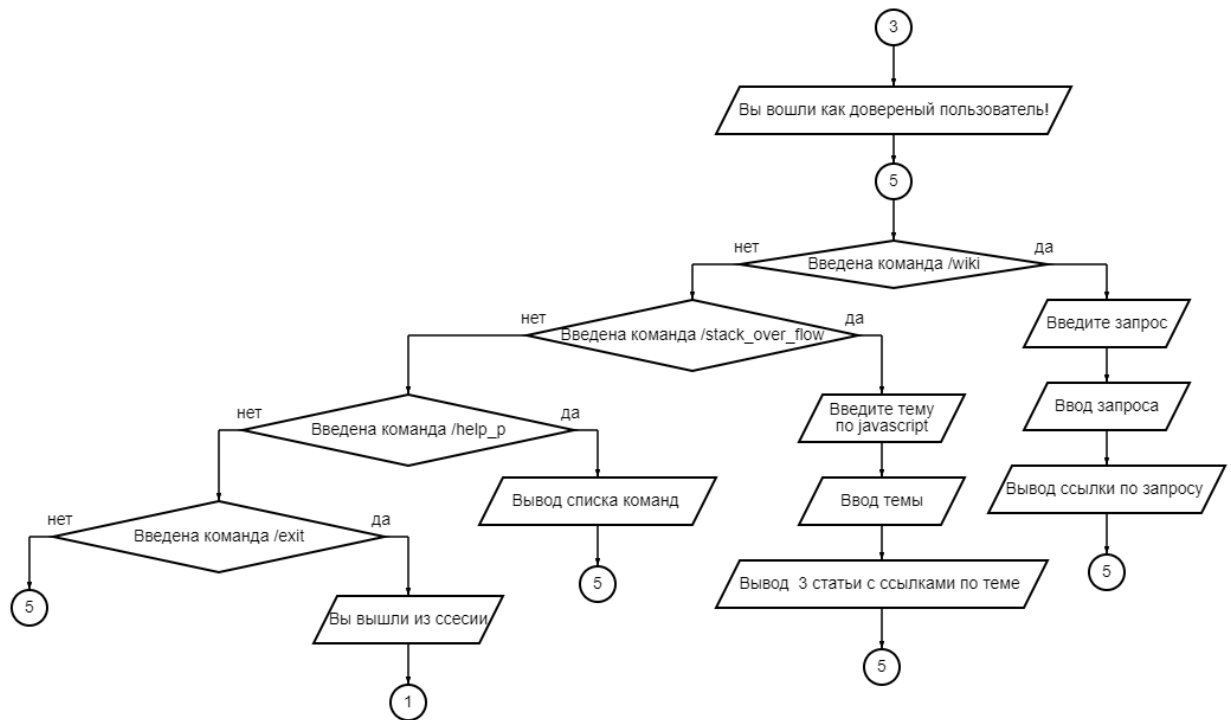


Рис. 2.2 Блок-схема алгоритма программы.

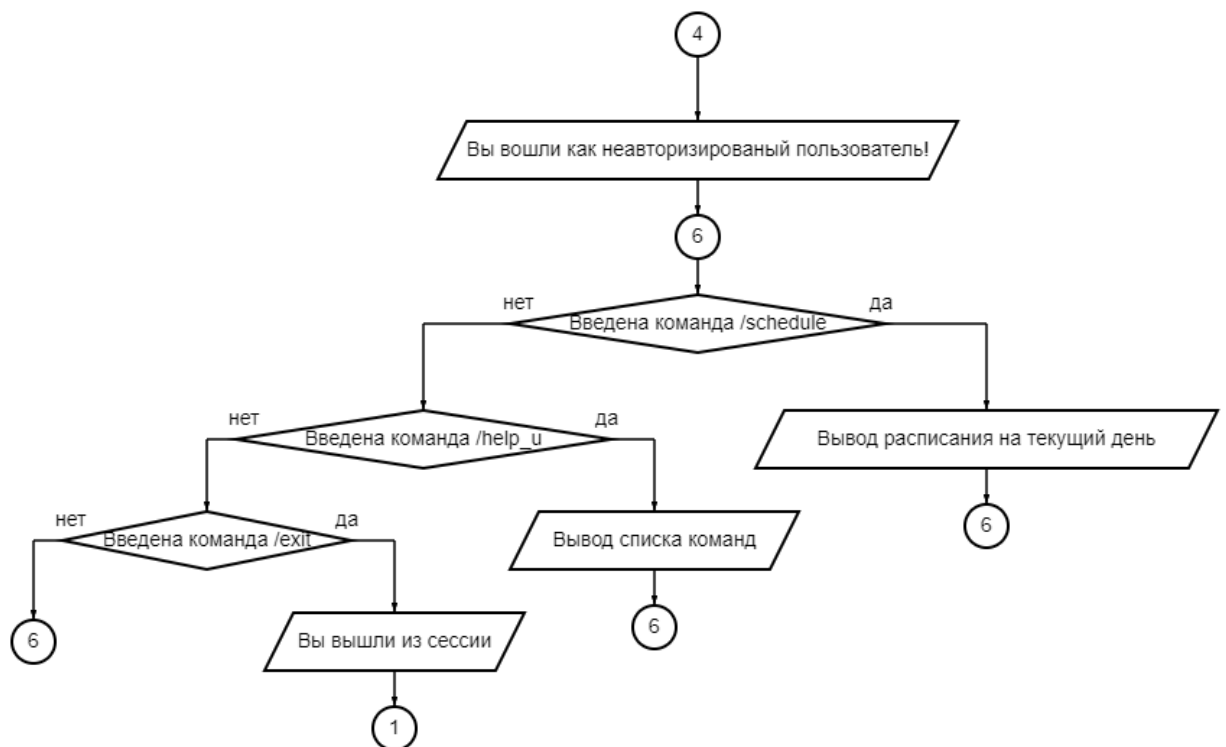


Рис. 2.3 Блок-схема алгоритма программы.

2.6. Проверка работоспособности программы.

При запуске бота нас встречает пустой чат с кнопкой меню(см. **ПРИЛОЖЕНИЕ** на стр. 19).

При вводе команды /start, выводится сообщение от бота и появляется клавиатура с командами(см. **ПРИЛОЖЕНИЕ** на стр. 20).

При нажатии на кнопку /about, выводится сообщения о боте (см. **ПРИЛОЖЕНИЕ** на стр. 20).

При нажатии на кнопку /my_id, выводится id пользователя(см. **ПРИЛОЖЕНИЕ** на стр. 21).

При нажатии на кнопку /admin, Выполняется вход в сессию админа, также на данном скриншоте я проверил работу всех команд доступных админу, их описания также приведено на скриншоте с помощью команды /help_a (см. **ПРИЛОЖЕНИЕ** на стр. 22).

Далее при нажатии на кнопку /exit, мы возвращаемся к начальной клавиатуре, выходя из сессии админа(см. **ПРИЛОЖЕНИЕ** на стр. 23).

При нажатии на кнопку /user_private, мы входим в сессию доверенного пользователя, на скриншоте я также привёл примеры выполнения все команд(см. **ПРИЛОЖЕНИЕ** на стр. 24).

Команда /help_p, выводит список команд, доступные доверенным пользователям.(см. **ПРИЛОЖЕНИЕ** на стр. 25).

При нажатии на кнопку /next, мы входим в сессию обычного пользователя, также на данном скриншоте, я проверил все доступные команды.(см. **ПРИЛОЖЕНИЕ** на стр. 26).

ЗАКЛЮЧЕНИЕ

В результате выполнения работы был разработан Telegram Бот, который предоставляет пользователям полезные функции: получение информации и ссылки на статьи по запросу пользователя.

Основные возможности бота:

1. Режим администратора.
2. Режим авторизованного пользователя.
3. Режим обычного пользователя.
4. Вывод расписания группы на текущий день.
5. Вывод ссылки на сайт Wikipedia, по запросу пользователя.
6. Вывод трёх поисковых запросов с сайта stackoverflow по теме JavaScript.
7. Авторизовать и деавторизовать пользователя в режиме администратора.

В ходе разработки бота были изучены и применены библиотеки Python: requests, aiogram, sqlalchemy, BeautifulSoup, datetime, selenium, что позволило реализовать его функциональность и обеспечить интеграцию с базой данных. Созданный бот является примером практического применения Python в области разработки приложений для мессенджеров, которые могут быть полезны для широкого круга пользователей. Дальнейшее развитие и расширение функциональности бота может включать в себя добавление новых возможностей, таких как интеграция с другими сервисами, а также совершенствование пользовательского интерфейса и повышение удобства использования.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Aiogram [Электронный ресурс] aiogram — асинхронный фреймворк для Telegram Bot API. Режим доступа: <https://aiogram.dev/>, свободный. - Загл. с экрана. — Яз. рус., англ. — (Дата обр. 12.07.2024).
2. Sublimetext [Электронный ресурс] Документация Sublimetext. Режим доступа: <https://sublimetext.ru/documentation>, свободный. - Загл. с экрана. — Яз. рус., англ. — (Дата обр. 26.07.2024).
3. GitHub [Электронный ресурс] Википедия — свободная энциклопедия. Режим доступа: <https://ru.wikipedia.org/wiki/GitHub>, свободный. - Загл. с экрана. — Яз. рус., англ. — (Дата обр. 28.06.2024).
4. BotFather [Электронный ресурс] Документация BotFather. Режим доступа: <https://core.telegram.org/bots/tutorial>, свободный. - Загл. с экрана. — Яз. англ. — (Дата обр. 13.07.2024).
5. Sqlalchemy [Электронный ресурс] Документация sqlalchemy. Режим доступа: <https://docs.sqlalchemy.org/en/20/>, свободный. - Загл. с экрана. — Яз. рус. — (Дата обр. 15.07.2024).
6. BeautifulSoup [Электронный ресурс] Документация BeautifulSoup. Режим доступа: <https://www.crummy.com/BeautifulSoup/bs4/doc.ru/bs4ru.html>, свободный. - Загл. с экрана. — Яз. англ. — (Дата обр. 15.07.2024).
7. requests [Электронный ресурс] Документация requests. Режим доступа: <https://pythonru.com/biblioteki/kratkoe-rukovodstvo-po-biblioteke-python-requests>, свободный. - Загл. с экрана. — Яз. рус. — (Дата обр. 21.07.2024).
8. Wikipedia [Электронный ресурс] Википедия — свободная энциклопедия. Режим доступа: <https://ru.wikipedia.org/wiki>, свободный. - Загл. с экрана. — Яз. рус. — (Дата обр. 22.07.2024).
9. Json [Электронный ресурс] Документация json. Режим доступа: <https://www.json.org/json-ru.html>, свободный. - Загл. с экрана. — Яз. рус., англ. — (Дата обр. 29.07.2024).

10. Datatime [Электронный ресурс] Документация Datatime. Режим доступа: <https://docs.python.org/datetime.html>, свободный. - Загл. с экрана. – Яз. рус., англ. — (Дата обр. 30.07.2024).
11. Selenium [Электронный ресурс] Документация selenium. Режим доступа: <https://www.selenium.dev/documentation/>, свободный. - Загл. с экрана. – Яз. рус., англ. — (Дата обр. 30.07.2024).
12. MyGitHub [Электронный ресурс] код бота. Режим доступа: <https://github.com/Zhc1561/Tg-bot>, Режим доступа: свободный. - Загл. с экрана. – Яз. рус., англ. — (Дата обр. 30.09.2024).

ПРИЛОЖЕНИЕ

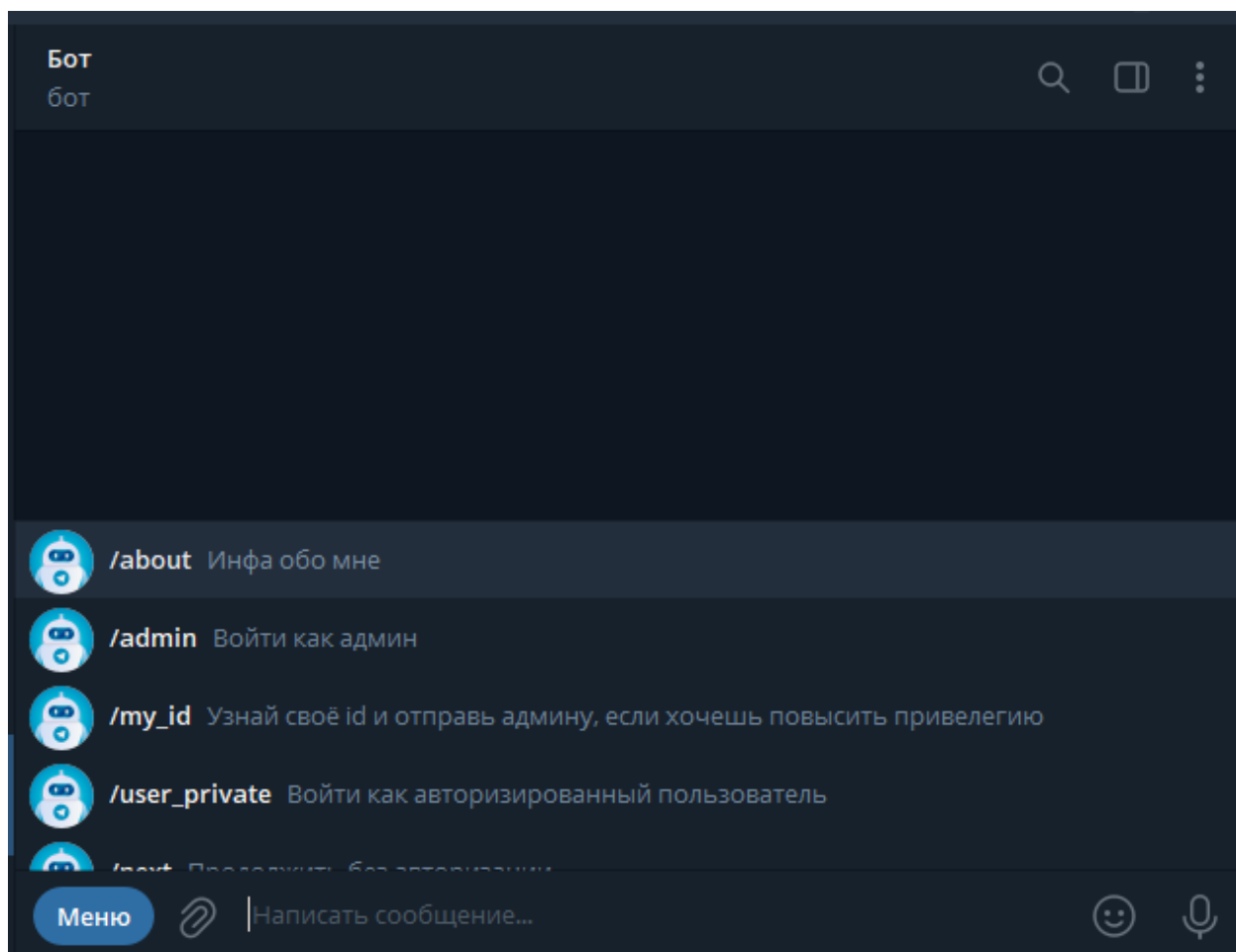


Рис. 4 Проверка работоспособности программы.

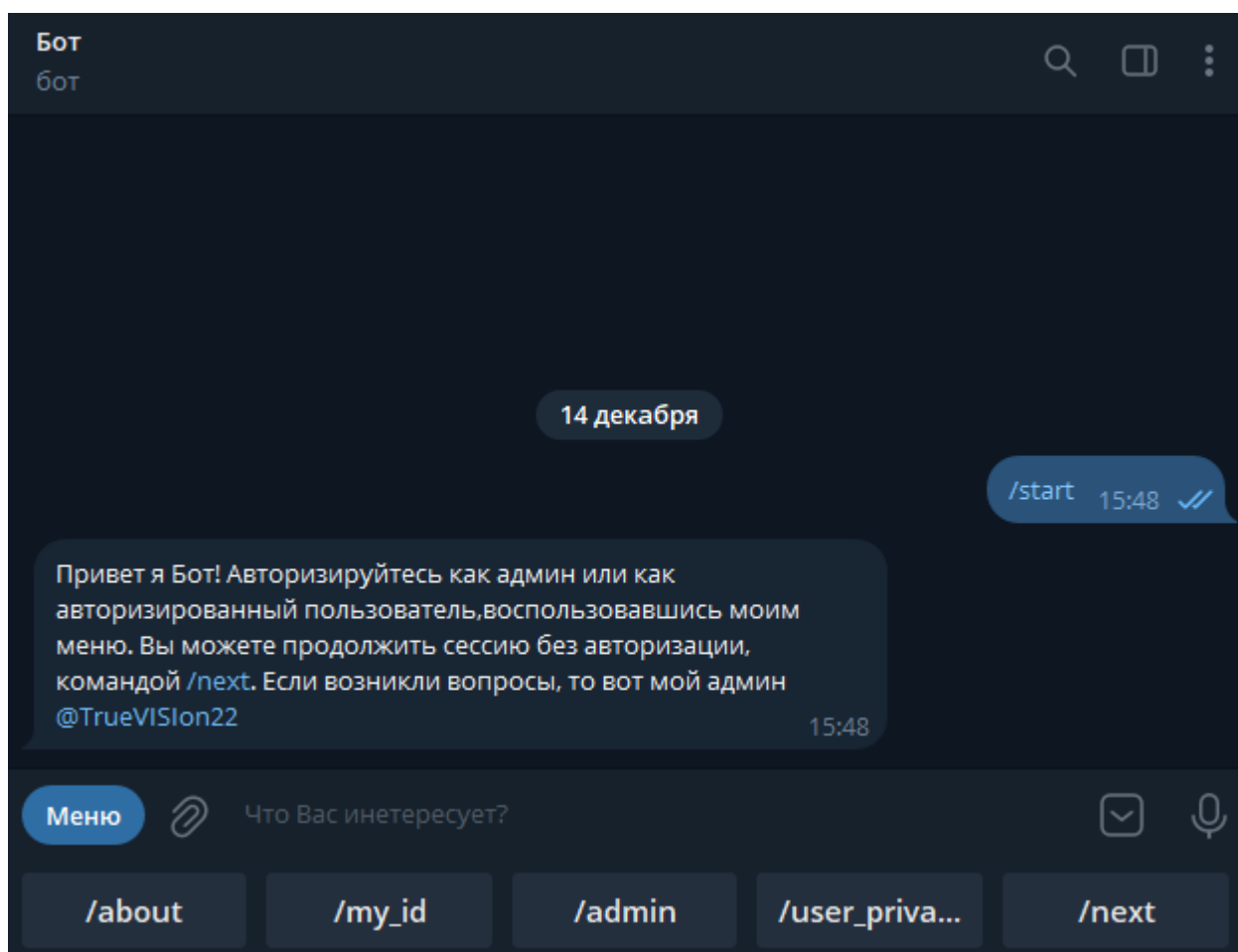


Рис. 5 Проверка работоспособности программы.

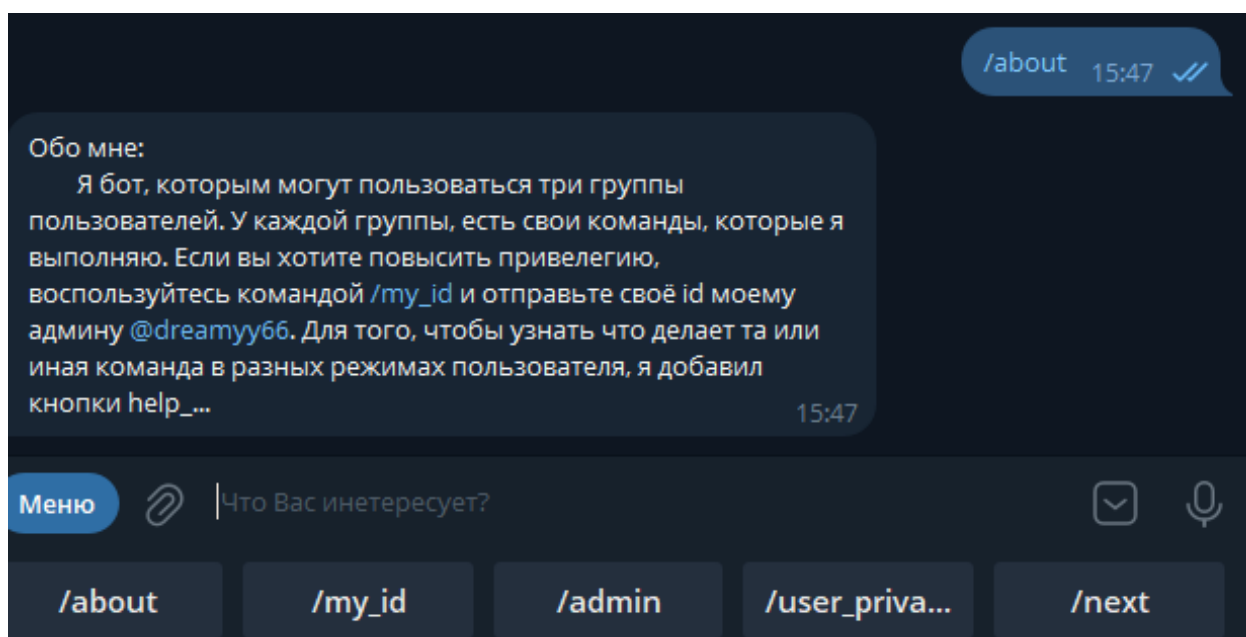


Рис. 6 Проверка работоспособности программы.

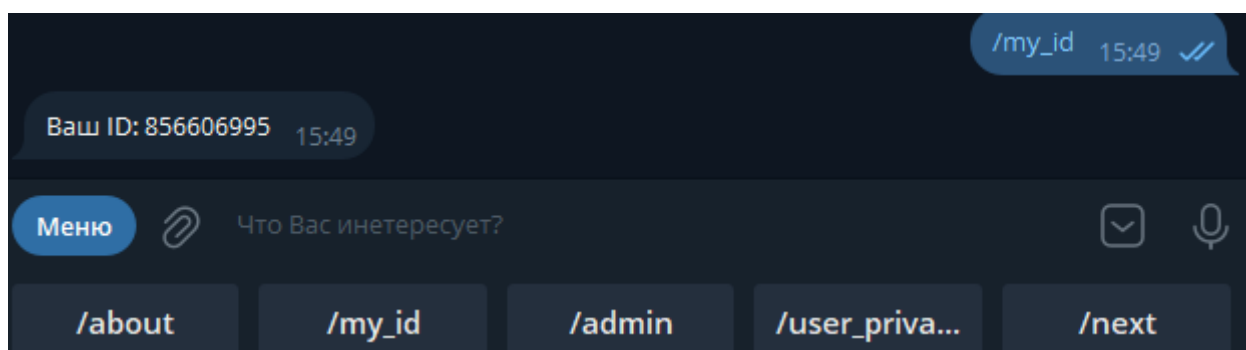


Рис. 7 Проверка работоспособности программы.

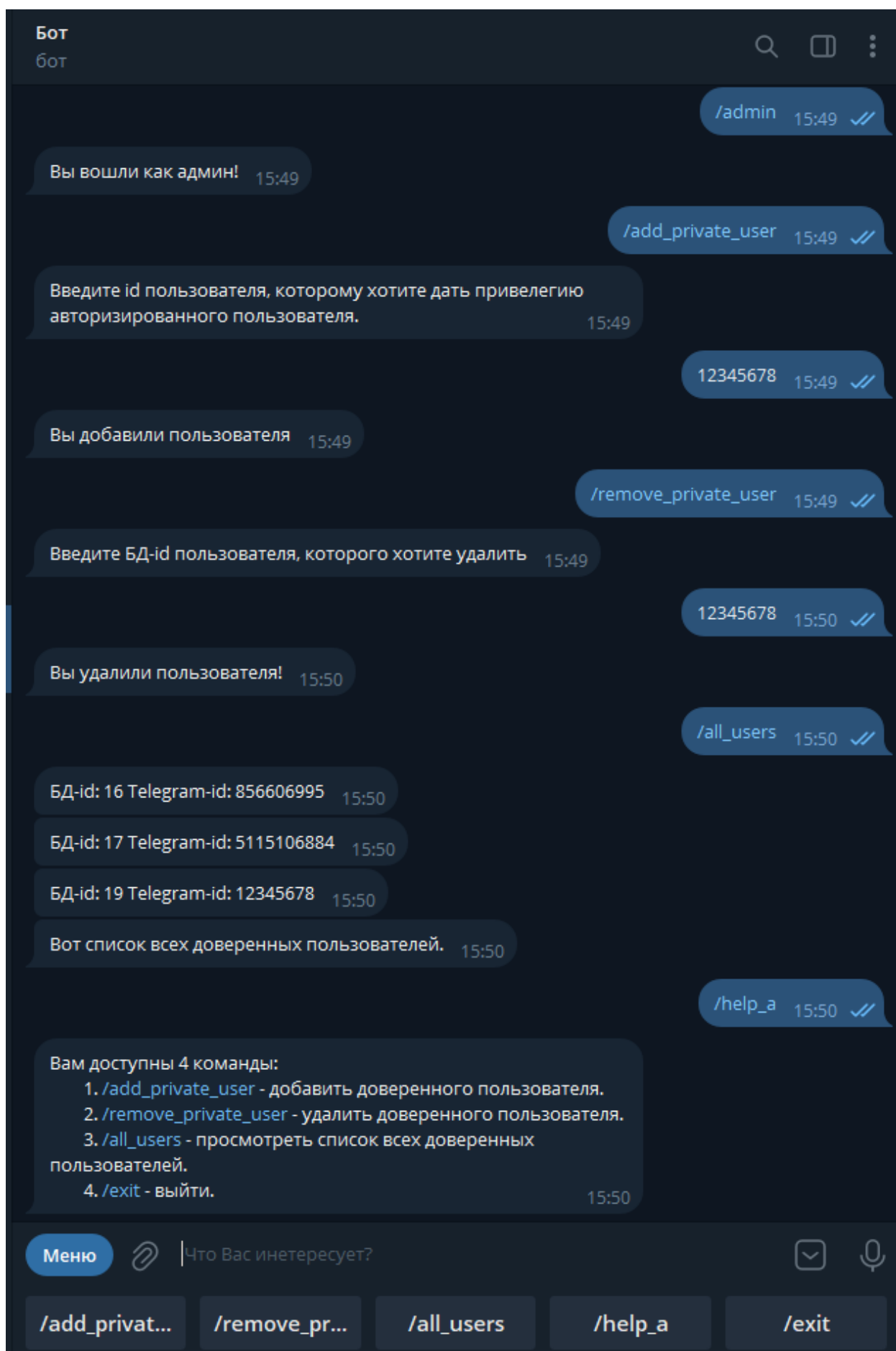


Рис. 8 Проверка работоспособности программы.

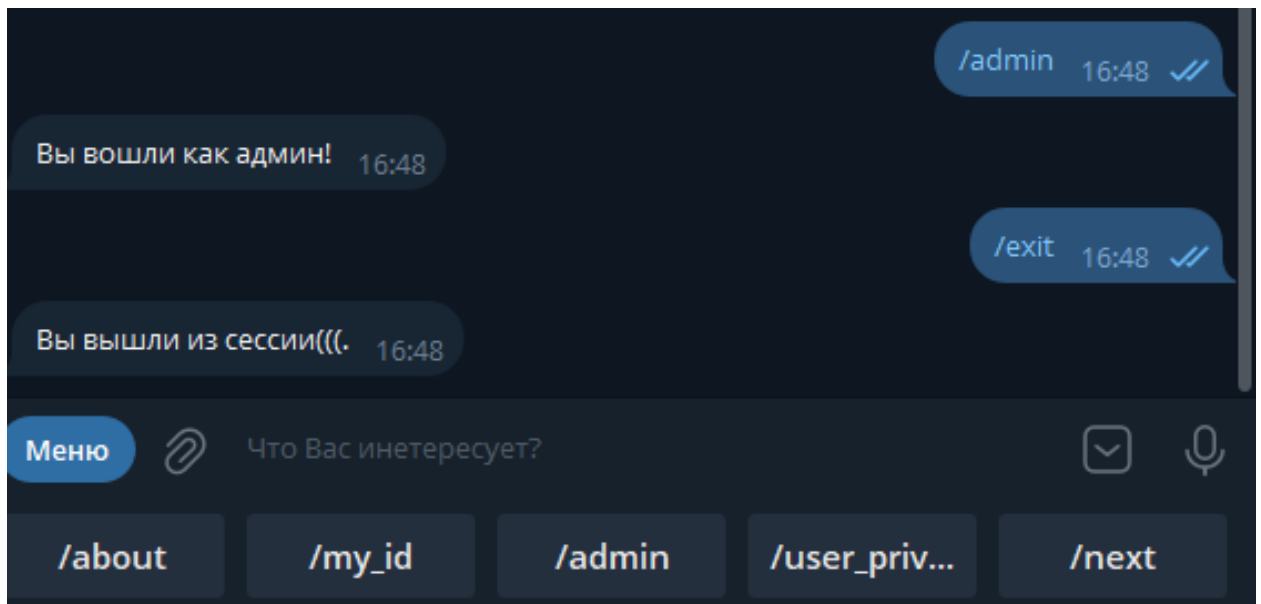


Рис. 9 Проверка работоспособности программы.

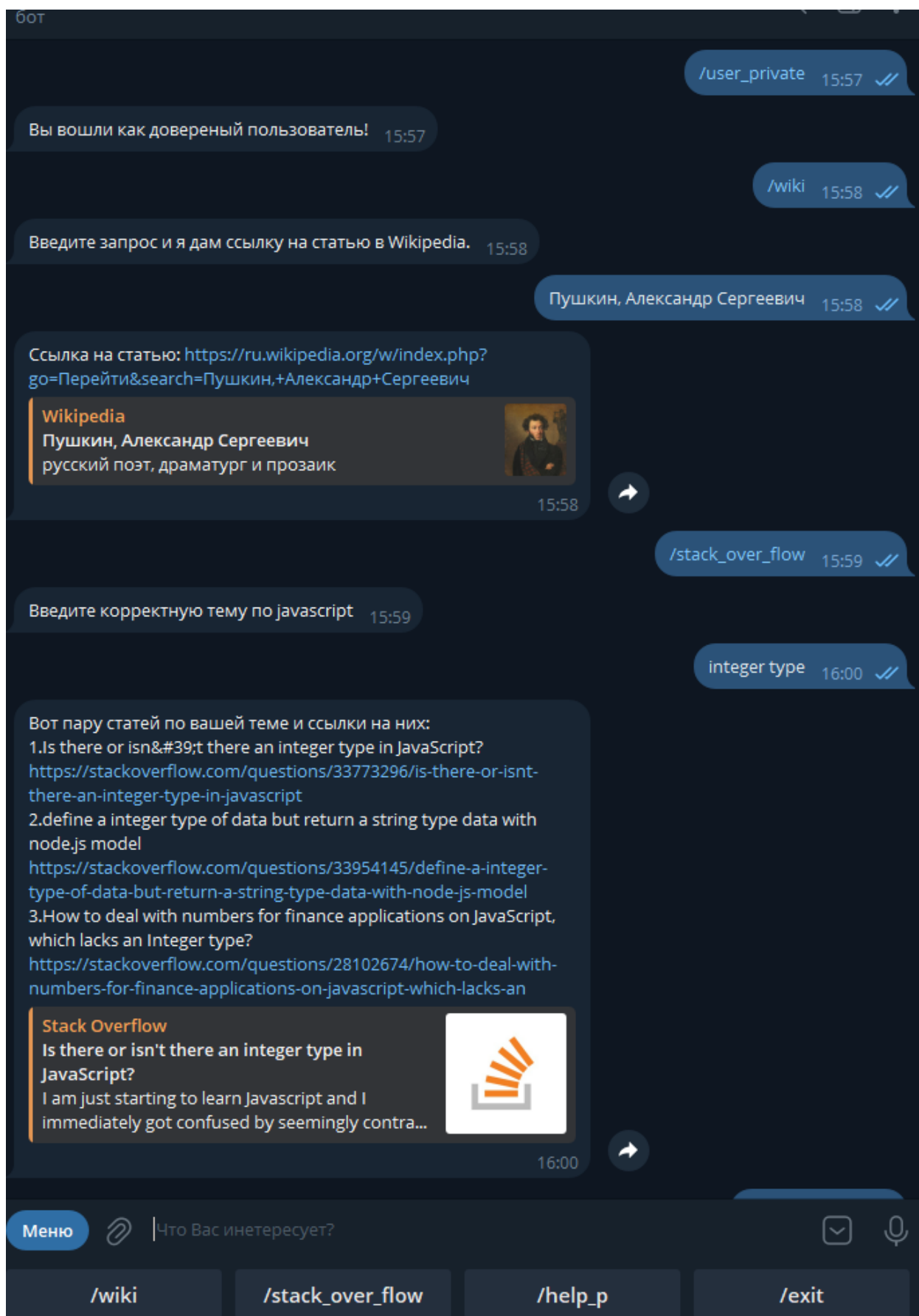


Рис. 10 Проверка работоспособности программы.

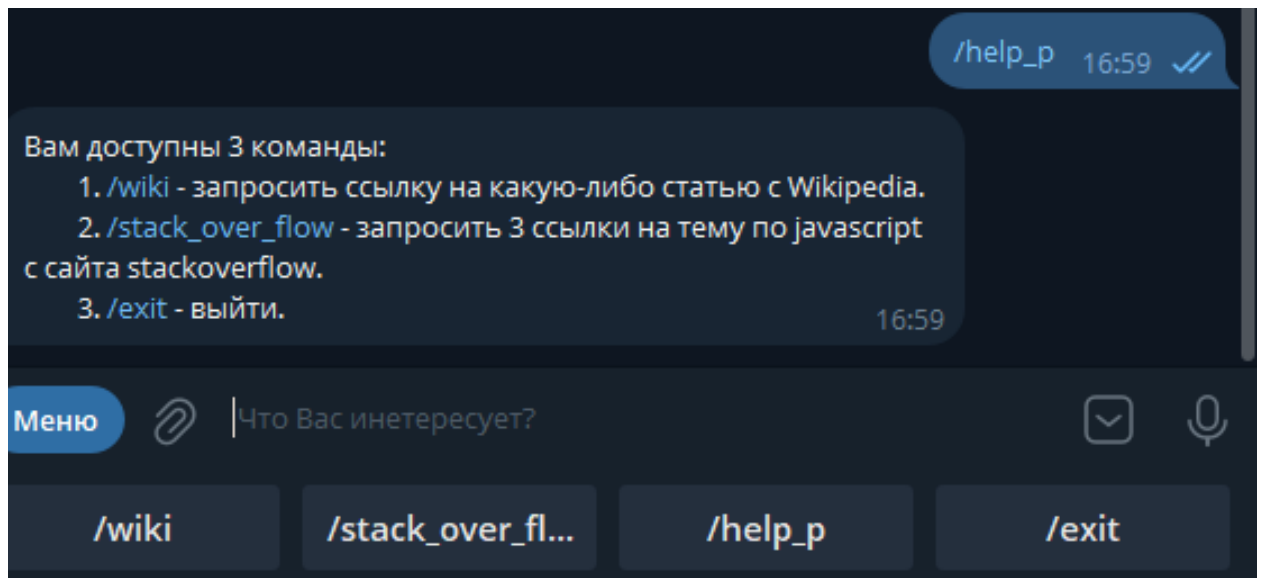


Рис. 11 Проверка работоспособности программы.

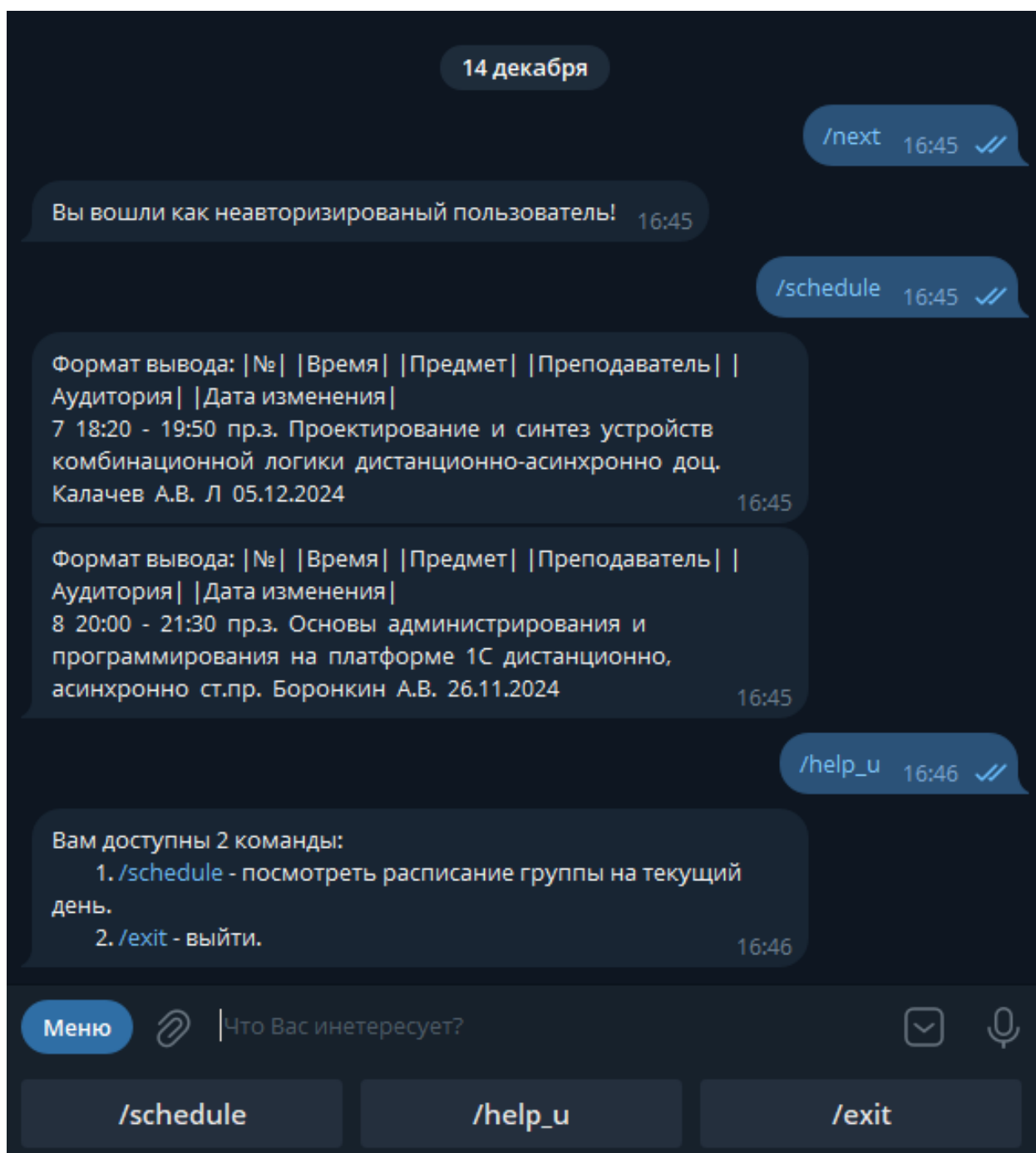


Рис. 12 Проверка работоспособности программы.

Главный файл Telegram бота на языке *Python* с использованием библиотеки *Aiogram*. Весь код можно доступен на моём github [12].

```

1 import os
2 import asyncio
3
4 from aiogram import Bot, Dispatcher, types
5 from aiogram.filters import CommandStart
6
7 from dotenv import find_dotenv, load_dotenv
8 load_dotenv(find_dotenv())
9
10 from middlewares.db import DataBaseSession
11
12 from database.engine import create_db, drop_db, session_maker
13
14 from handlers.admin_private import admin_router
15
16 from handlers.user_private import user_private_router
17
18 from handlers.privelegy_user import privat_router
19
20 from common.bot_cmds_list import private
21
22 ALLOWED_UPDATES = ['message', 'edited_message']
23
24 bot = Bot(token = os.getenv('TOKEN'))
25 bot.my_admins_list = [856606995,]
26 dp = Dispatcher()
27
28 dp.include_router(user_private_router)
29 dp.include_router(admin_router)
30 dp.include_router(privat_router)
31 async def on_startup(bot):
32     run_param = False
33     if run_param:
34         await drop_db()
35         await create_db()
36
37 async def on_shutdown(bot):
38     print('Бот лёр')
39
40 async def main():
41     dp.startup.register(on_startup)
42     dp.shutdown.register(on_shutdown)
43
44     dp.update.middleware(DataBaseSession(session_pool=session_maker))
45
46     await bot.delete_webhook(drop_pending_updates=True)
47     await bot.set_my_commands(commands=private, scope = types.BotCommandScopeAllPrivateChats())

```

```
48     await dp.start_polling(bot, allowed_updates=ALLOWED_UPDATES)
49
50 asyncio.run(main())
```
